`f4_fp`: from 13–24× behind msolve to level with it

2026-10-12. Branch `f4-fp-msolve-gap`. Run report: `reports/run-20261011T2359Z/report.pdf`. Prior comparison: `research/gb_cas_comparison_20261011` (PR #1675).

Question

#1675 measured `f4_fp`, the repository's $\mathbb{F}_p$ F4, at 12–24× slower than msolve on square generic systems; msolve is the open-source F4 from Faugère's group. Where does the time go, and how much of the gap closes without changing any basis?

Changes, in the order measured

change	control	effect where first measured (same binary)
Gebauer–Möller criteria B, M, F on top of the product criterion. Pairs above the degree bound are held, not dropped, so the criteria can retire them, and `pairs_above_bound` counts only real truncation.	`F4_FP_GM=0`	quad_n9 3397 ⟶ 472 ms (7.2×). The basis is complete at degree 10, and the old steps at degrees 11–14 only reduced redundant pairs to zero. Largest matrix 15 164 ⟶ 2 034 rows.
Final interreduction as one matrix step: symbolic preprocessing, Faugère–Lachartre, reduced echelon form.	`F4_FP_LINEAR_INTERREDUCE=0`	quad_n9 459 ⟶ 304 ms. Polynomial interreduction had been 43 % of the run.
Packed monomials: 8-bit fields of a `u128` for grevlex with ≤ 15 variables and degree < 128. Products are adds; divisibility, lcm and coprimality are guarded word operations, including inside Gebauer–Möller.	`F4_FP_PACKED=0`	quad_n9 303 ⟶ 170 ms (4 threads)
Symbolic preprocessing hashes each term once and records its column id, so there is no second hashing pass to build the matrix.	—	quad_n10 1.19×, quad_n11 1.17× (1 thread)
A dedicated hasher for packed keys, plus round-by-round parallel symbolic preprocessing. `FxHasher` copied a `u128` byte by byte and left the bucket bits unmixed.	`F4_FP_SYMBOLIC_PAR=0`	symbolic preprocessing of one quad_n11 step 151 ⟶ 49 ms; quad_n11 1.25× at 4 threads

Every change leaves the reduced basis identical: fingerprints match on all 16 systems, and Singular or msolve confirms each basis (run report).

Result (run-20261011T2359Z: 3 interleaved rounds, medians)

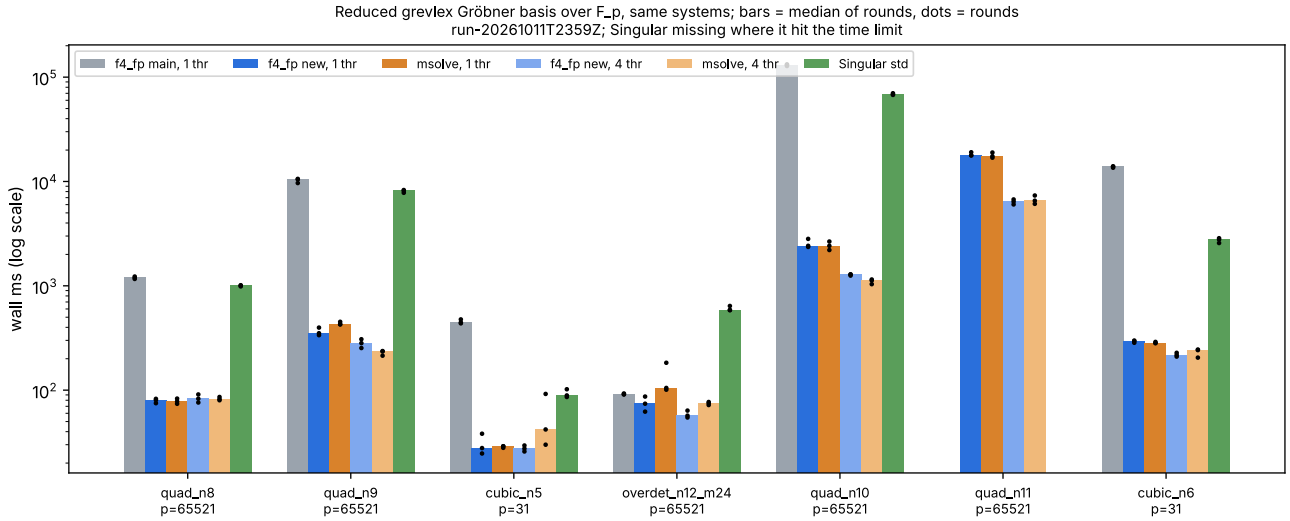


Figure 1: Reduced grevlex Gröbner bases over $\mathbb{F}_p$, same systems, run-20261011T2359Z.

case	main ÷ new (1 thr)	msolve ÷ new (1 thr)	msolve ÷ new (4 thr)
quad_n8_p65521	14.9	0.97	0.99

quad_n9_p65521	29.8	1.22	0.83
cubic_n5_p31	16.0	1.04	1.52
overdet_n12_m24_p65521	1.2	1.42	1.32
quad_n10_p65521	54.1	1.00	0.88
quad_n11_p65521	not run	0.98	1.02
cubic_n6_p31	47.0	0.96	1.11

Ratios are derived from the medians in the run report; above 1 means the new `f4_fp` is faster.

- Against `main`, single-threaded: 15–54× on the square systems.
- Against msolve: level single-threaded (0.96–1.42 on the systems above 50 ms). At 4 threads it ranges from 0.83 to 1.52; msolve is ahead on quad_n9 and quad_n10, and we are ahead on cubic_n5, cubic_n6 and the overdetermined systems.
- Against Singular `std`: 3.2× (cubic_n5) to 28× (quad_n10) faster on the square systems.

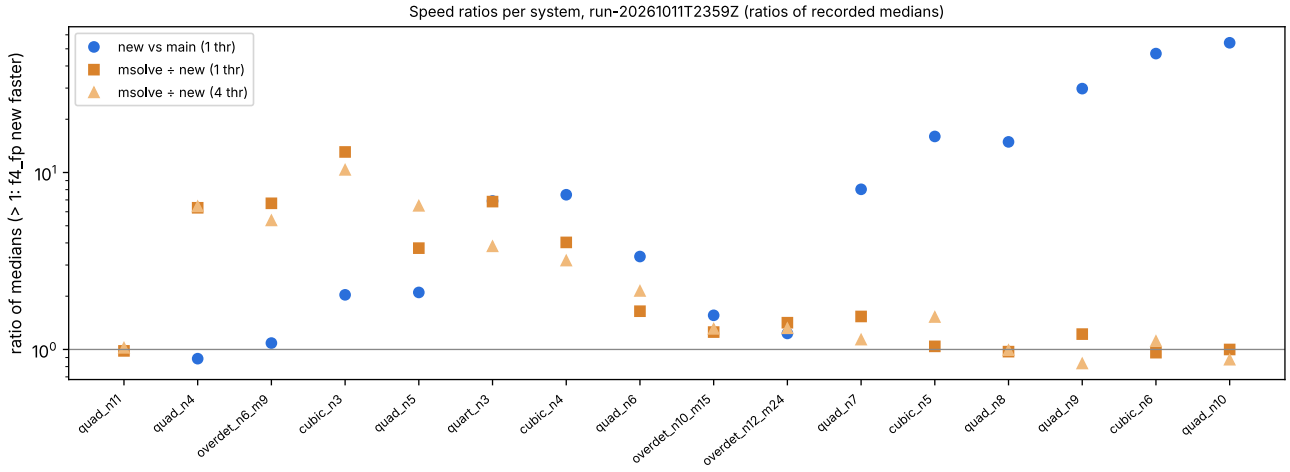


Figure 2: Ratios for every system. msolve's ratios on the smallest systems mostly measure its process start-up.

Scope and limits

One shared host (isolation L0, a different host model from the earlier runs), 3 rounds, random dense systems over $\mathbb{F}_{31}$ to $\mathbb{F}_{65521}$. msolve is timed with its process start-up; Singular and our builds are timed in-process. Level with msolve is not "fastest": Magma was not run (scripts in #1675), and these are generic systems only. This is an $\mathbb{F}_p$ engine result; it does not touch the Boolean index-calculus engines or any IC-versus-rho comparison. Graphs checked and left unchanged: `docs/ic/progress-timeline.json`, `docs/index-calculus-scoreboard.html`, `docs/performance-gains/`.

Where the time goes now, and what was tried

From the `F4_DEBUG` phase timers during development (not part of the recorded run): at 4 threads on quad_n11 the reduction is about 75 % of the run: building `B'` 31 %, the pair-row update 36 %, and the dense residue plus conversions 13 %. About 58 % of the pair rows still reduce to zero; those are syzygies the Gebauer–Möller criteria do not see.

- **Next:** signature (F5-style) criteria to skip those rows; a cheaper final interreduction (9 %); better thread scaling of the `B'` build.
- **Tried and not kept:**
  - cache-tiling the Faugère–Lachartre update: level or slower in 4 interleaved rounds, so the kernel is not bandwidth-bound at these sizes;
  - choosing the sparsest reducer: identical matrices;
  - packed symbolic preprocessing in the final interreduction: no measurable change.